Objective: In this lab We need to design latch and see the waveforms of the output as well as the RTL tool view and technology map view to understand the relationship between input and output.

Part 1: In this part We use the VHDL code of SR latch and see the output of latches in waveform and also the technology view. The screenshots are provided below:

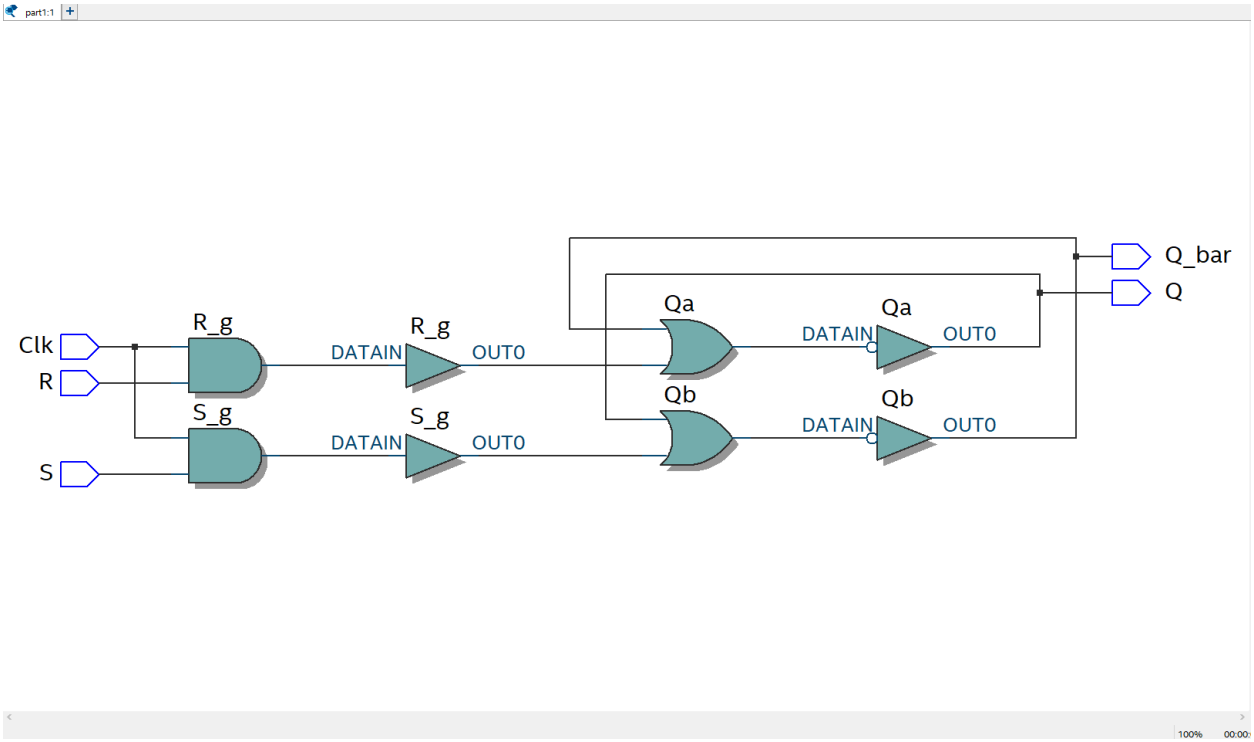


Figure 1.1: RTL viewer tool

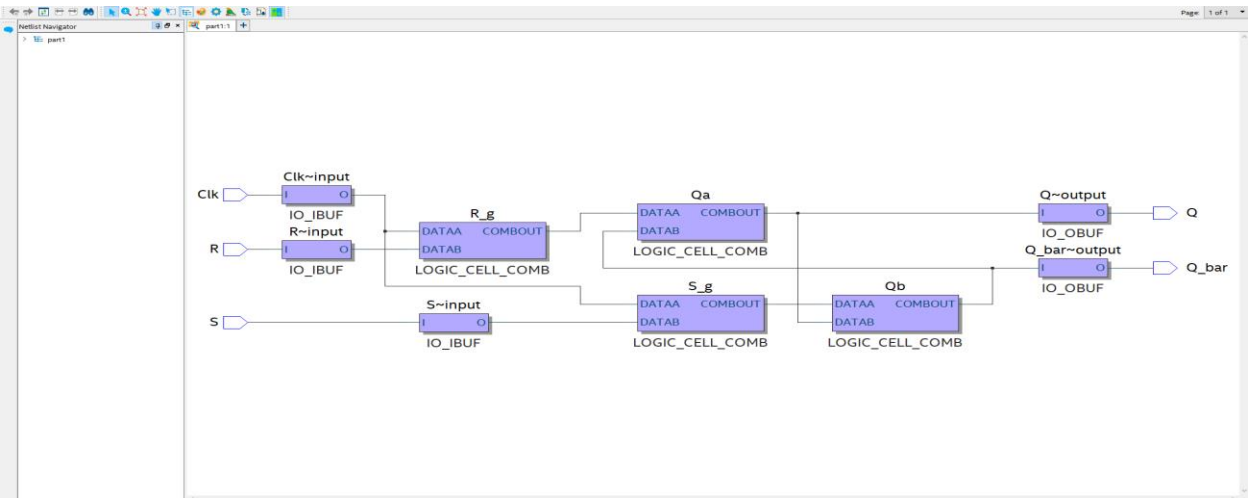


Figure 1.2: Technology map viewer tool

part1.vhd — Compilation Report - part1

```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.all;
3
4   ENTITY part1 IS
5   PORT ( Clk, R, S : IN STD_LOGIC;
6   Q,Q_bar : OUT STD_LOGIC);
7   END part1;
8
9   ARCHITECTURE Structural OF part1 IS
10
11  SIGNAL R_g, S_g, Qa, Qb : STD_LOGIC ;
12  --ATTRIBUTE KEEP : BOOLEAN;
13  --ATTRIBUTE KEEP OF R_g, S_g, Qa, Qb : SIGNAL IS TRUE;
14
15  BEGIN
16  R_g <= R AND Clk;
17  S_g <= S AND Clk;
18  Qa <= NOT (R_g OR Qb);
19  Qb <= NOT (S_g OR Qa);
20  Q <= Qa;
21  Q_bar <= Qb;
22  END Structural;
```

Figure 1.3: RS latch VHDL code



Figure 1.4: Generated waveform

Part 2: In this part 2, I use the VHDL code of D latch and extend the d-flipflop for multiple waveform to see the output of d-latches in waveform and also the technology view. The screenshots are provided below:

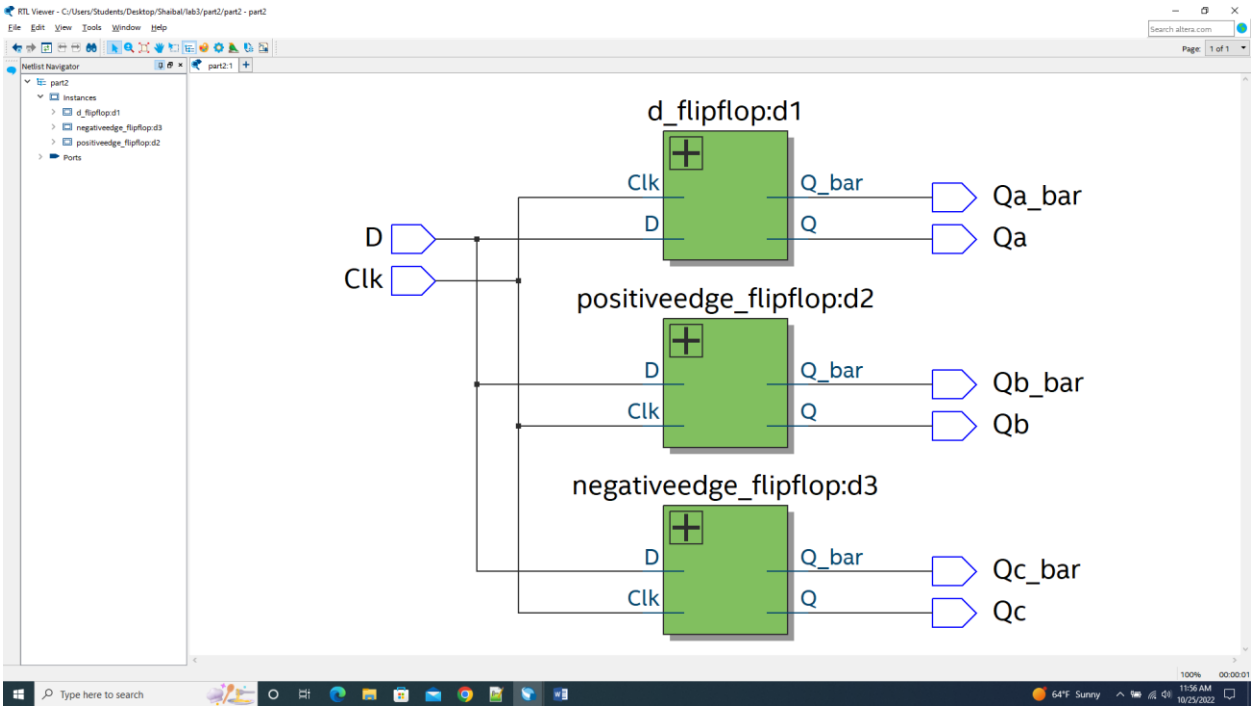


Figure 2.1: RTL viewer tool

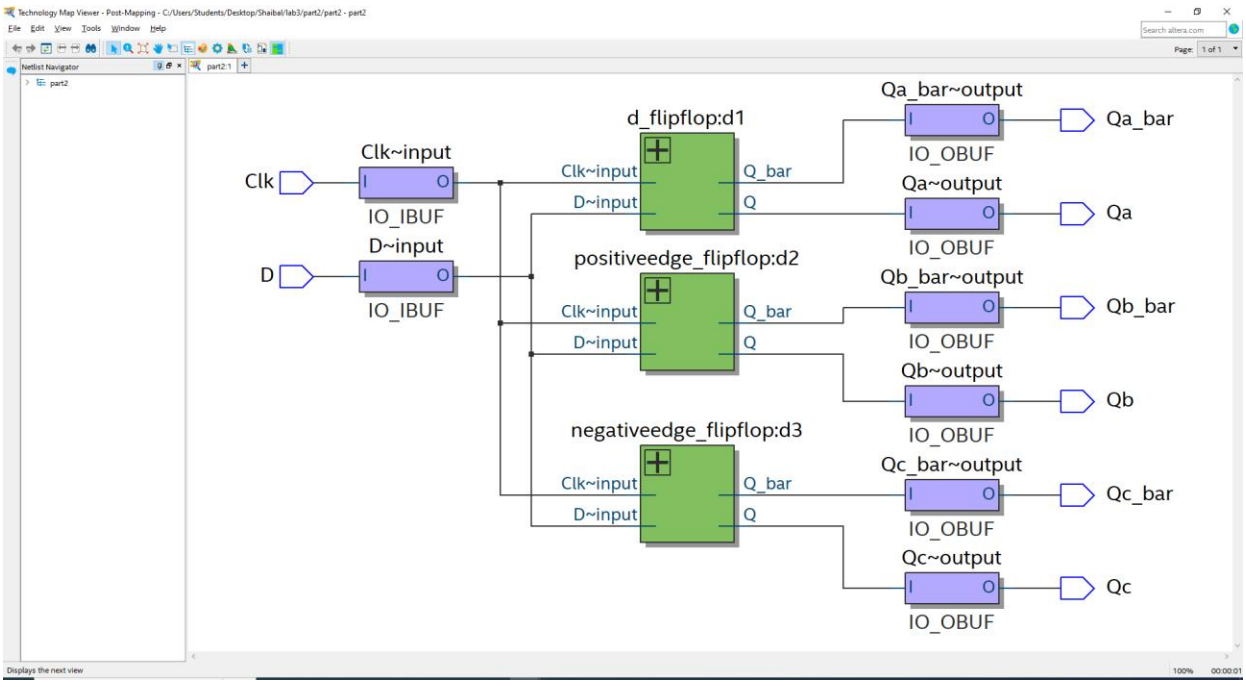


Figure 2.2: Technology map viewer tool

part2.vhd | d_flipflop.vhd | Compilation Report - part2 | positiveedge_flipflop.vhd | negativeedge_flipflop.vhd

```
--Shaibal Saha

LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY part2 IS

PORT ( D, Clk : IN STD_LOGIC ;
       Qa,Qa_bar,Qb,Qb_bar,Qc,Qc_bar : OUT STD_LOGIC) ;

END part2 ;

ARCHITECTURE Behavior OF part2 IS

component d_flipflop
      PORT ( D, Clk : IN STD_LOGIC ;
             Q,Q_bar : OUT STD_LOGIC) ;
END component ;

component positiveedge_flipflop
      PORT ( D, Clk : IN STD_LOGIC ;
             Q,Q_bar : OUT STD_LOGIC) ;
END component ;

component negativeedge_flipflop
   PORT ( D, Clk : IN STD_LOGIC ;
          Q,Q_bar : OUT STD_LOGIC) ;
END component ;

BEGIN

 d1: d_flipflop port map(D,clk,Qa, Qa_bar);
 d2: positiveedge_flipflop port map(D,clk,Qb, Qb_bar);
 d3: negativeedge_flipflop port map(D,not(clk),Qc, Qc_bar);

END Behavior ;
```

Figure 2.3: Code

part2.vhd | d_flipflop.vhd | Compilation Report - part2 | positiveedge_flipflop.vhd | negativeedge_flipflop.vhd

```
--Shaibal Saha

LIBRARY ieee ;
USE ieee.std_logic_1164.all ;


ENTITY d_flipflop IS
PORT ( D, Clk : IN STD_LOGIC ;
Q,Q_bar : OUT STD_LOGIC) ;
END d_flipflop ;

ARCHITECTURE Behavior OF d_flipflop IS
BEGIN

PROCESS ( D, Clk )
BEGIN
IF Clk = '1' THEN
Q<=D;
Q_bar<= not(D);
END IF ;
END PROCESS ;

END Behavior ;
```

Figure 2.4: Code

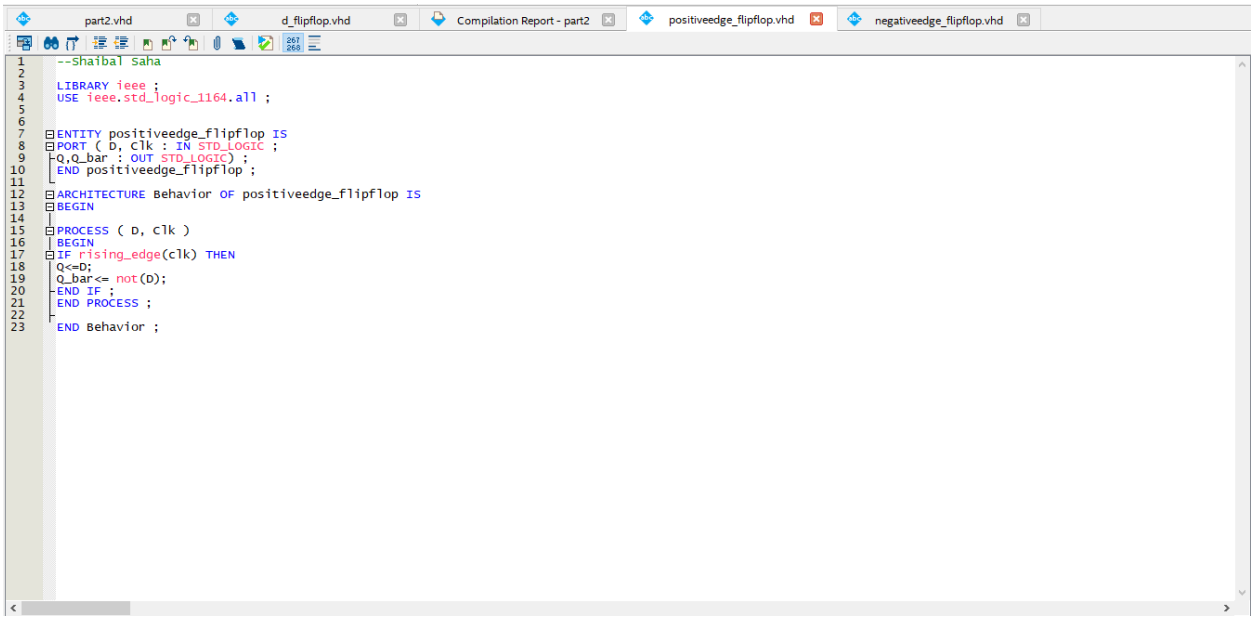


Figure 2.5: Code

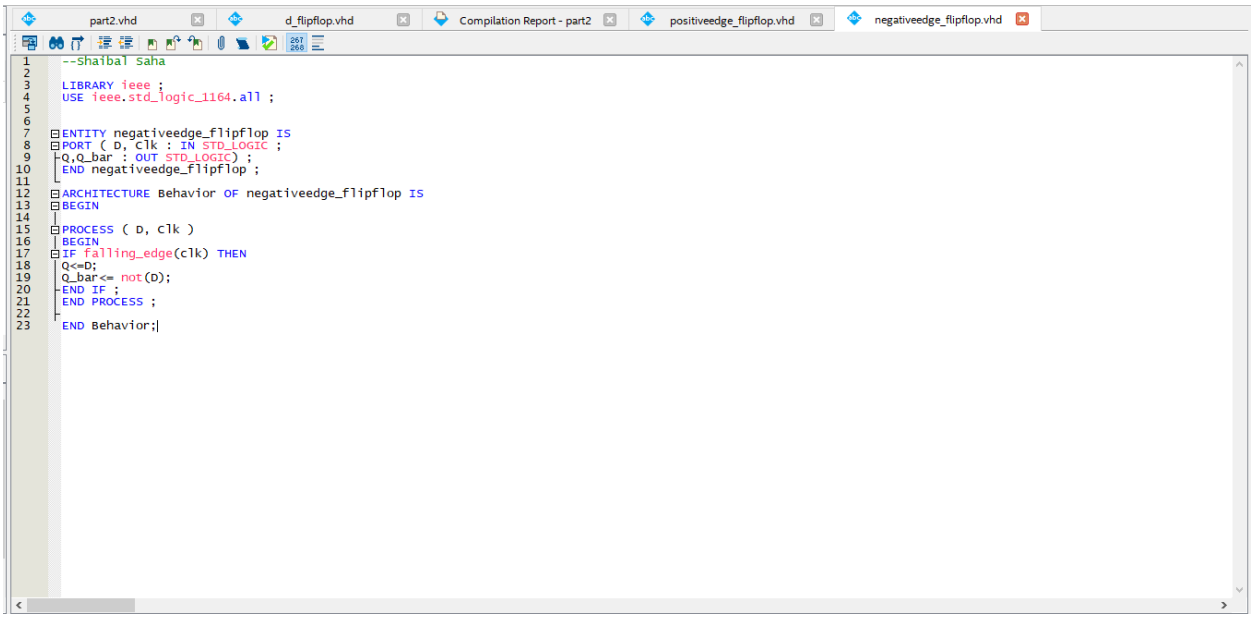


Figure 2.6: Code

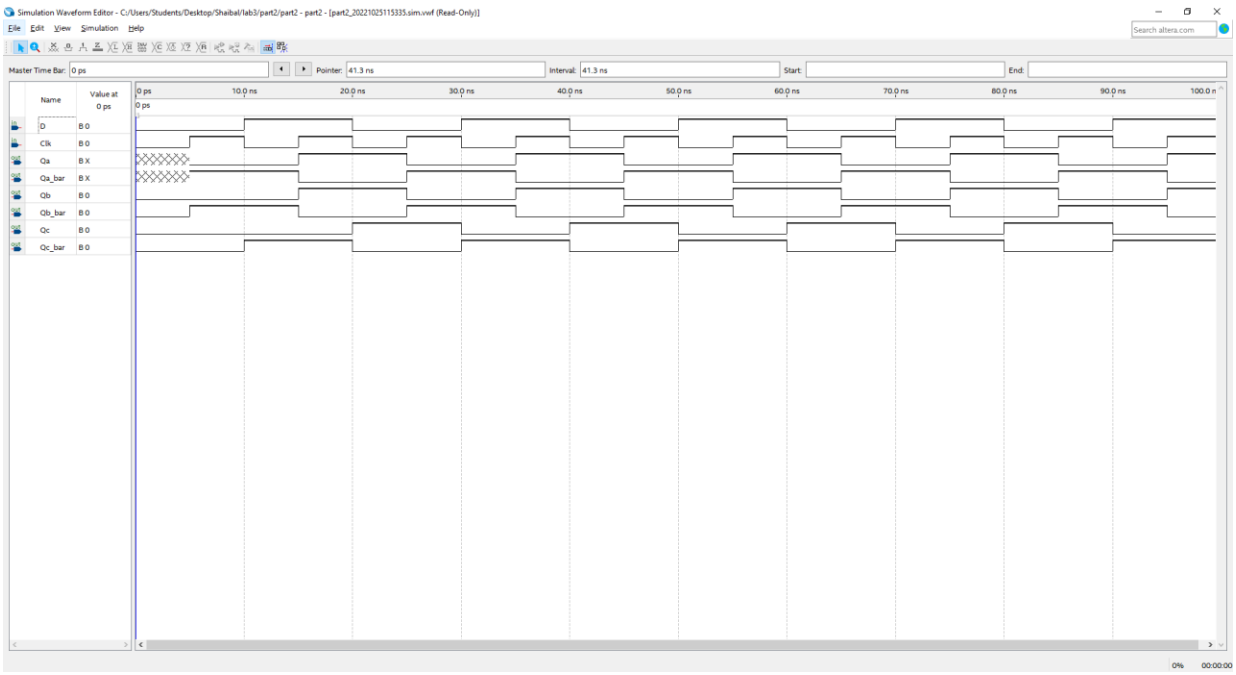


Figure 2.7: Generated waveform

Part 3: In this part 3, I add two 8 bit number using a key to store the variable A . I have to design bcd to binary for 8 bit to get perfectly in 7 segment display. I attached the code as well as the some of the screenshots below:

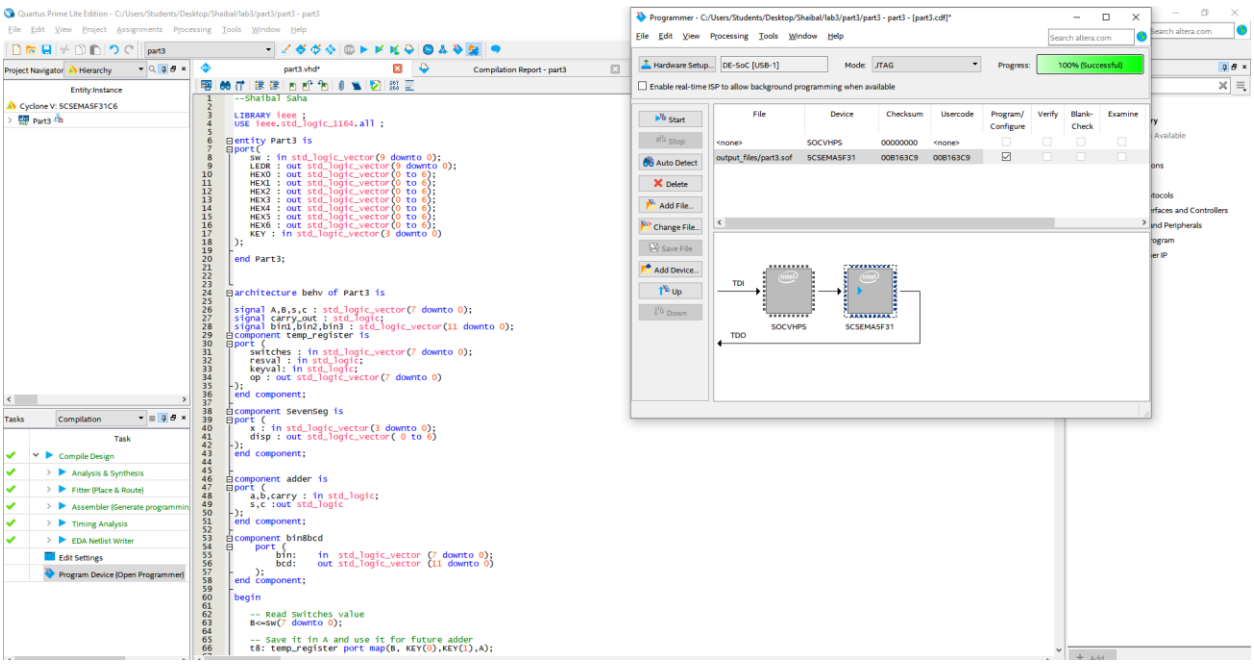
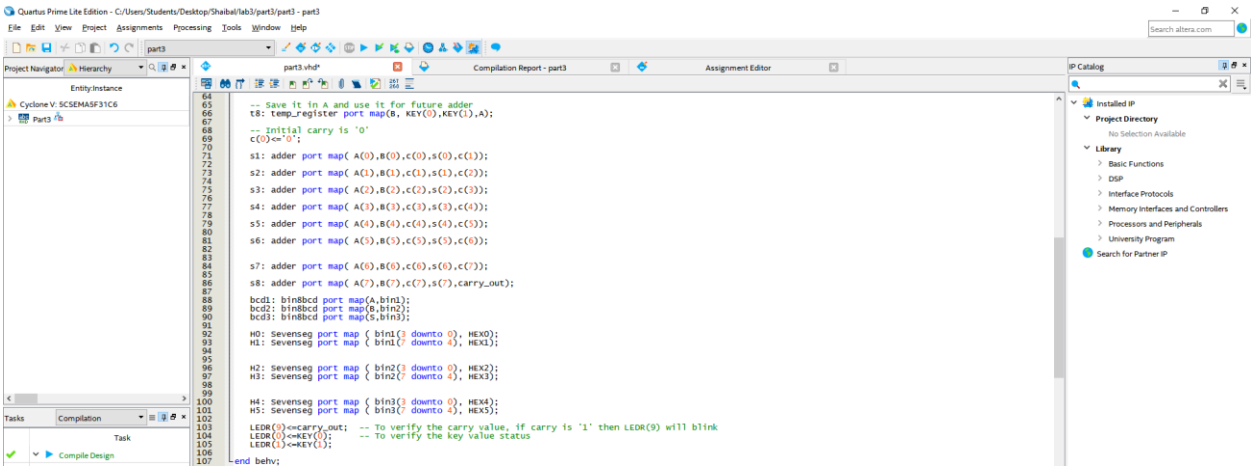


Figure 3.1: Code



```
    -- Save it in A and use it for future adder
    t8: temp_register port map(B, KEY(0),KEY(1),A);

    -- Initial carry is '0'
    c(0)<='0';

    s1: adder port map( A(0),B(0),c(0),s(0),c(1));

    s2: adder port map( A(1),B(1),c(1),s(1),c(2));

    s3: adder port map( A(2),B(2),c(2),s(2),c(3));

    s4: adder port map( A(3),B(3),c(3),s(3),c(4));

    s5: adder port map( A(4),B(4),c(4),s(4),c(5));

    s6: adder port map( A(5),B(5),c(5),s(5),c(6));


    s7: adder port map( A(6),B(6),c(6),s(6),c(7));

    s8: adder port map( A(7),B(7),c(7),s(7),carry_out);

    bcd1: bin8bcd port map(A,bin1);
    bcd2: bin8bcd port map(B,bin2);
    bcd3: bin8bcd port map(S,bin3);

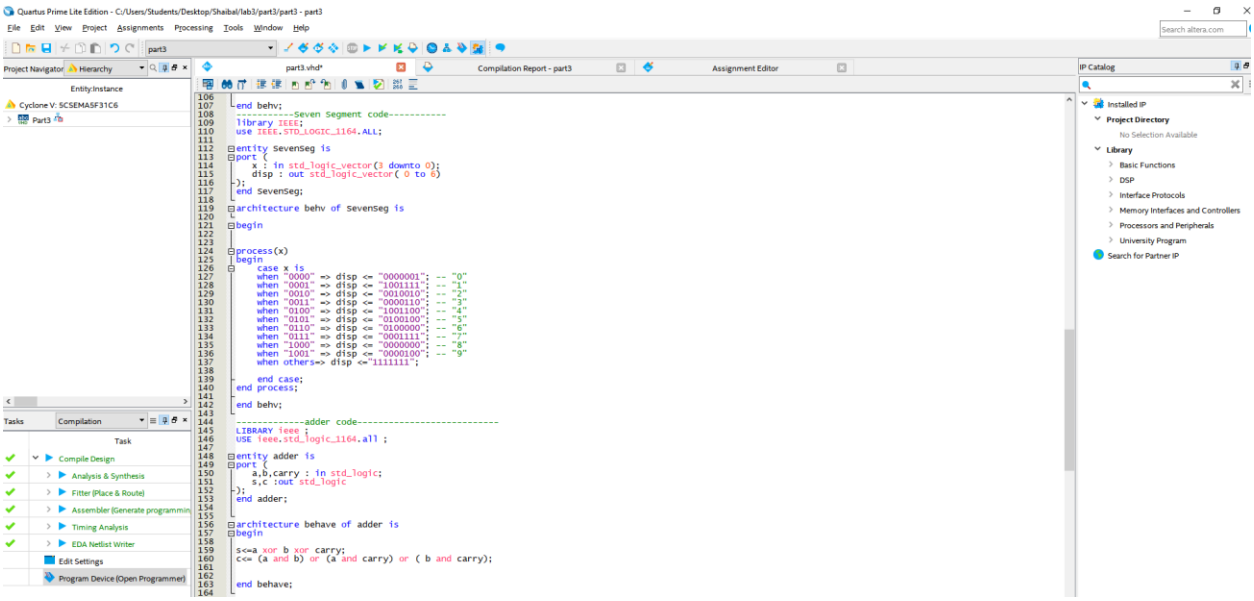
    H0: Sevenseg port map ( bin1(3 downto 0), HEX0);
    H1: Sevenseg port map ( bin1(7 downto 4), HEX1);


    H2: Sevenseg port map ( bin2(3 downto 0), HEX2);
    H3: Sevenseg port map ( bin2(7 downto 4), HEX3);


    H4: Sevenseg port map ( bin3(3 downto 0), HEX4);
    H5: Sevenseg port map ( bin3(7 downto 4), HEX5);

    LEDR(9)<=carry_out;  -- To verify the carry value, if carry is '1' then LEDR(9) will blink
    LEDR(0)<=KEY(0);     -- To verify the key value status
    LEDR(1)<=KEY(1);
end behv;
```

Figure 3.2: Code



```
end behv;
----------Seven Segment code----------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SevenSeg is
port (
    x : in std_logic_vector(3 downto 0);
    disp : out std_logic_vector( 0 to 6)
);
end SevenSeg;

architecture behv of SevenSeg is

begin

process(x)
begin
    case x is
    when "0000" => disp <= "0000001"; -- "0"
    when "0001" => disp <= "1001111"; -- "1"
    when "0010" => disp <= "0010010"; -- "2"
    when "0011" => disp <= "0000110"; -- "3"
    when "0100" => disp <= "1001100"; -- "4"
    when "0101" => disp <= "0100100"; -- "5"
    when "0110" => disp <= "0100000"; -- "6"
    when "0111" => disp <= "0001111"; -- "7"
    when "1000" => disp <= "0000000"; -- "8"
    when "1001" => disp <= "0000100"; -- "9"
    when others=> disp <="1111111";

    end case;
end process;
end behv;

----------adder code----------
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

entity adder is
port (
    a,b,carry : in std_logic;
    s,c :out std_logic
);
end adder;

architecture behave of adder is
begin

s<=a xor b xor carry;
c<= (a and b) or (a and carry) or ( b and carry);

end behave;
```

Figure 3.3: Code

```
----------Temporary Register code----------
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

Entity temp_register is
port (
    switches : in std_logic_vector(7 downto 0);
    resval : in std_logic;
    keyval: in std_logic;
    op : out std_logic_vector(7 downto 0)
);
end temp_register;

architecture behavior of temp_register is
begin

    process(keyval,resval)
    begin
    if(resval='0') then
     op<="00000000";

    elsif(keyval'event and keyval='0') then
     op<=switches;

    end if;
    end process;

end behavior;
```

Figure 3.4: Code

```
----------------------Binary to bcd -------------------------

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bin8bcd is
    port (
        bin:    in  std_logic_vector (7 downto 0);
        bcd:    out std_logic_vector (11 downto 0)
    );
end entity;

architecture struct of bin8bcd is
    procedure add3 (signal bin: in  std_logic_vector (3 downto 0);
                    signal bcd: out std_logic_vector (3 downto 0)) is
    variable is_gt_4:  std_logic;
    begin
        is_gt_4 := bin(3) or (bin(2) and (bin(1) or bin(0)));

        if is_gt_4 = '1' then
        -- if to_integer(unsigned (bin)) > 4 then
            bcd <= std_logic_vector(unsigned(bin) + "0011");
        else
            bcd <= bin;
        end if;
    end procedure;

    signal U0bin,U1bin,U2bin,U3bin,U4bin,U5bin,U6bin:
                std_logic_vector (3 downto 0);

    signal U0bcd,U1bcd,U2bcd,U3bcd,U4bcd,U5bcd,U6bcd:
                std_logic_vector (3 downto 0);
begin
    U0bin <= '0' & bin (7 downto 5);
    U1bin <= U0bcd(2 downto 0) & bin(4);
    U2bin <= U1bcd(2 downto 0) & bin(3);
    U3bin <= U2bcd(2 downto 0) & bin(2);
    U4bin <= U3bcd(2 downto 0) & bin(1);

    U5bin <= '0' & U0bcd(3) & U1bcd(3) & U2bcd(3);
    U6bin <= U5bcd(2 downto 0) & U3bcd(3);

U0: add3(U0bin,U0bcd);

U1: add3(U1bin,U1bcd);

U2: add3(U2bin,U2bcd);

U3: add3(U3bin,U3bcd);

U4: add3(U4bin,U4bcd);

U5: add3(U5bin,U5bcd);

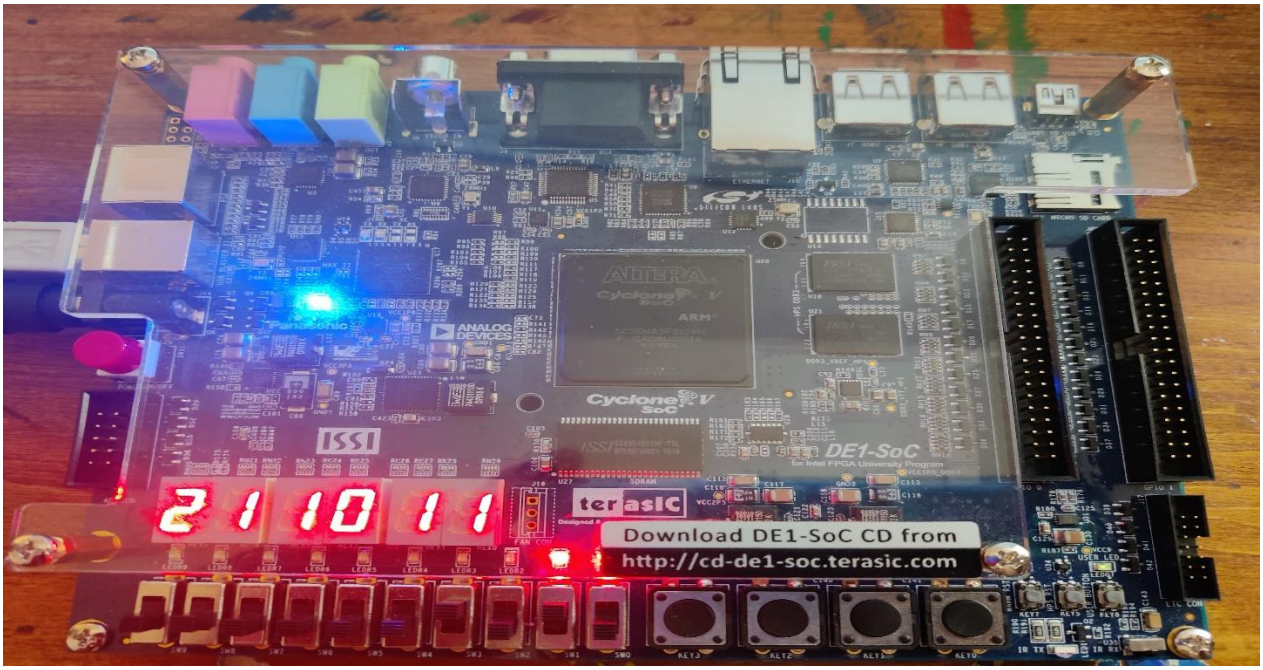
U6: add3(U6bin,U6bcd);

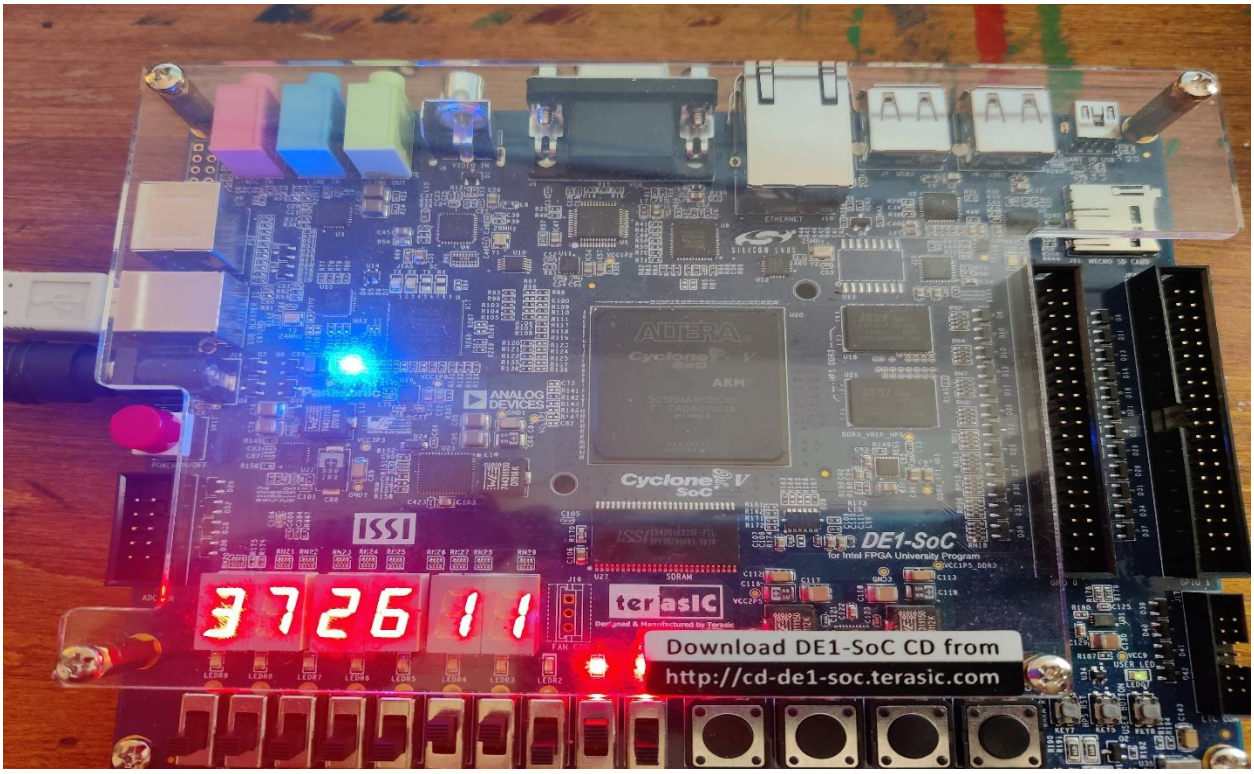
OUTP:
    bcd <= '0' & '0' & U5bcd(3) & U6bcd & U4bcd & bin(0);

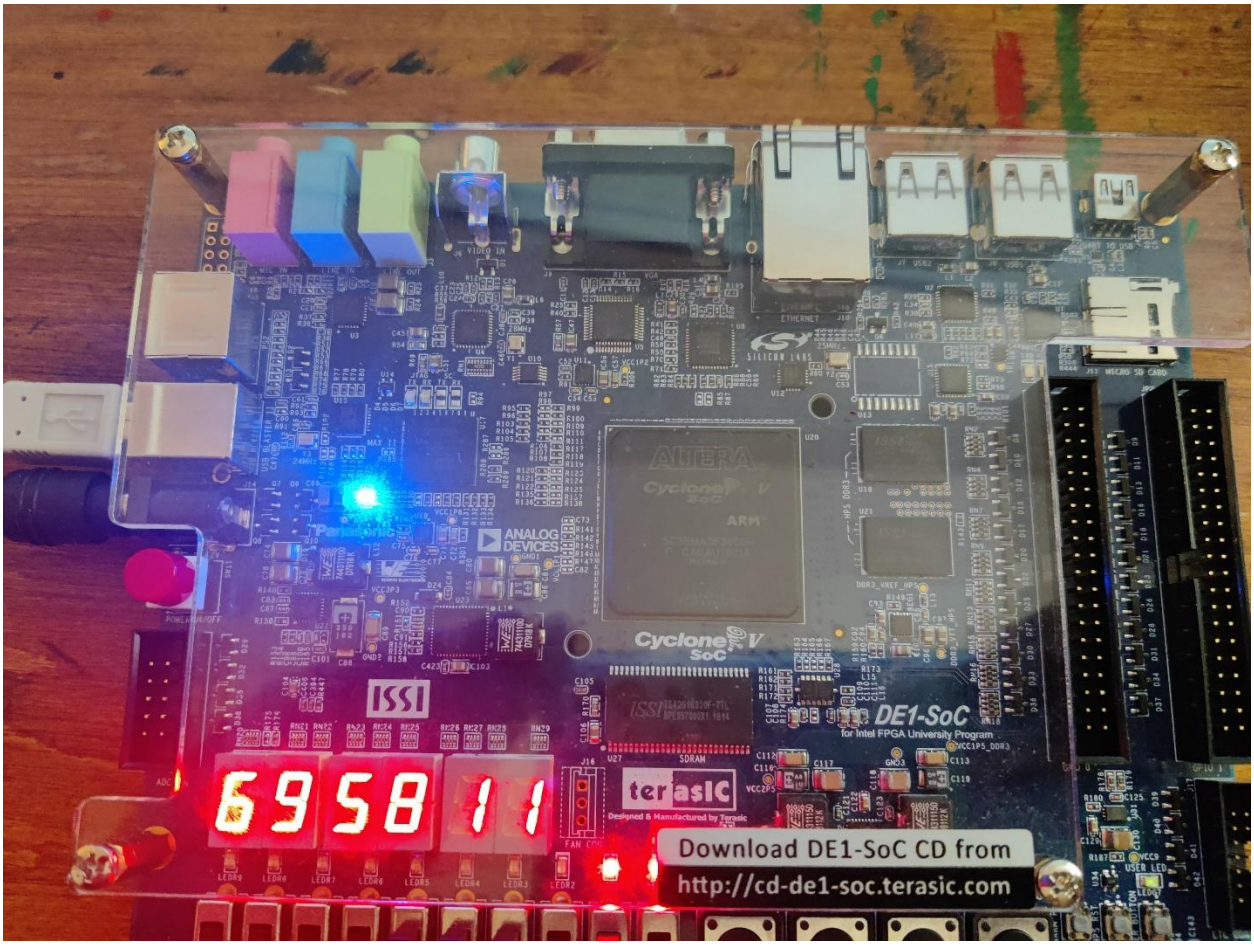
end architecture;
```

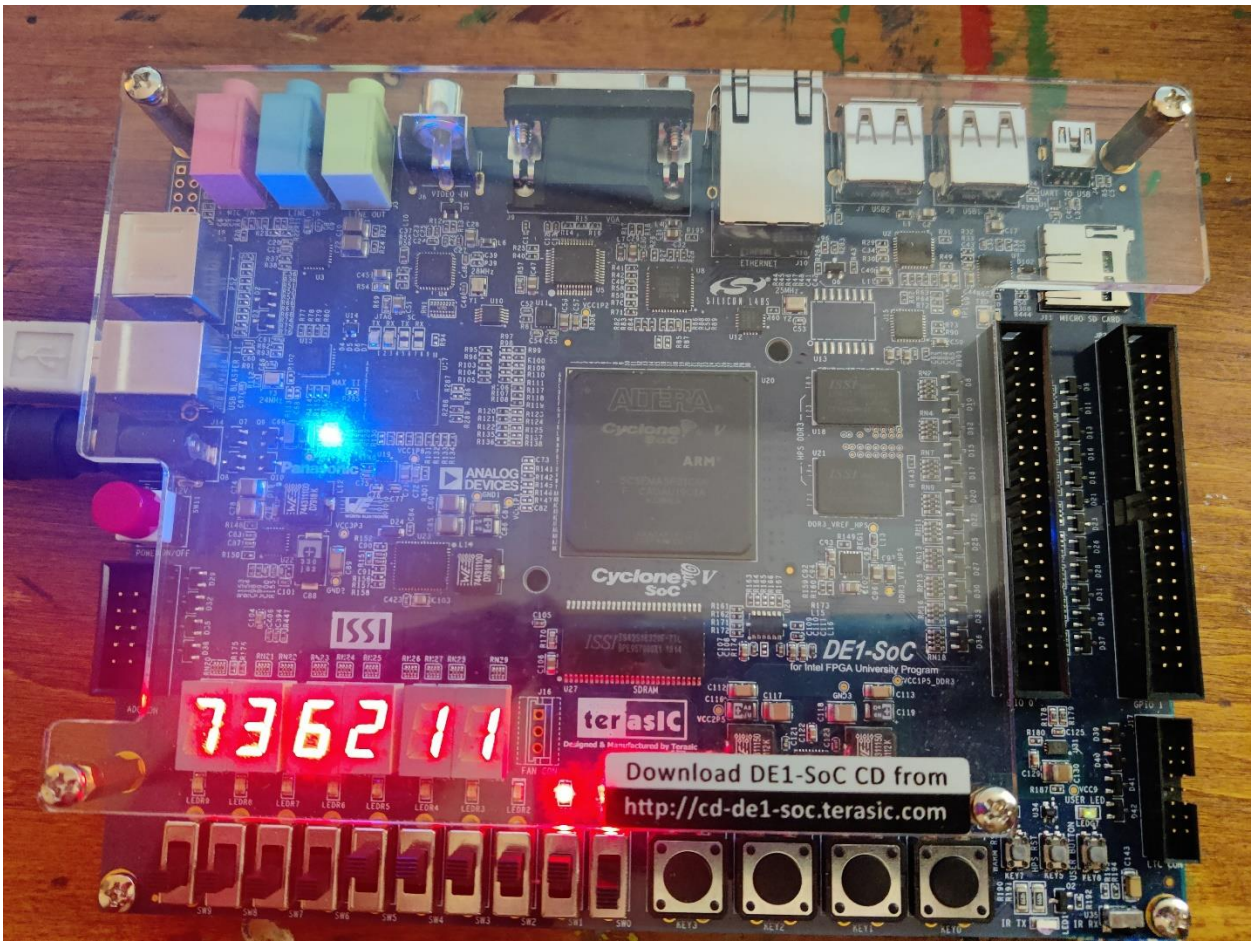
Figure 3.5: Code

Output: Output of the program are given below (some of the photos from the DE-soc1 board)




3726 11
Download DE1-SoC CD from
http://cd-de1-soc.terasic.com


6958 11
Download DE1-SoC CD from
http://cd-de1-soc.terasic.com


7362 11
Download DE1-SoC CD from
http://cd-de1-soc.terasic.com
DE1-SoC
for Intel FPGA University Program
Cyclone V SoC
terasIC
ANALOG DEVICES
ISSI

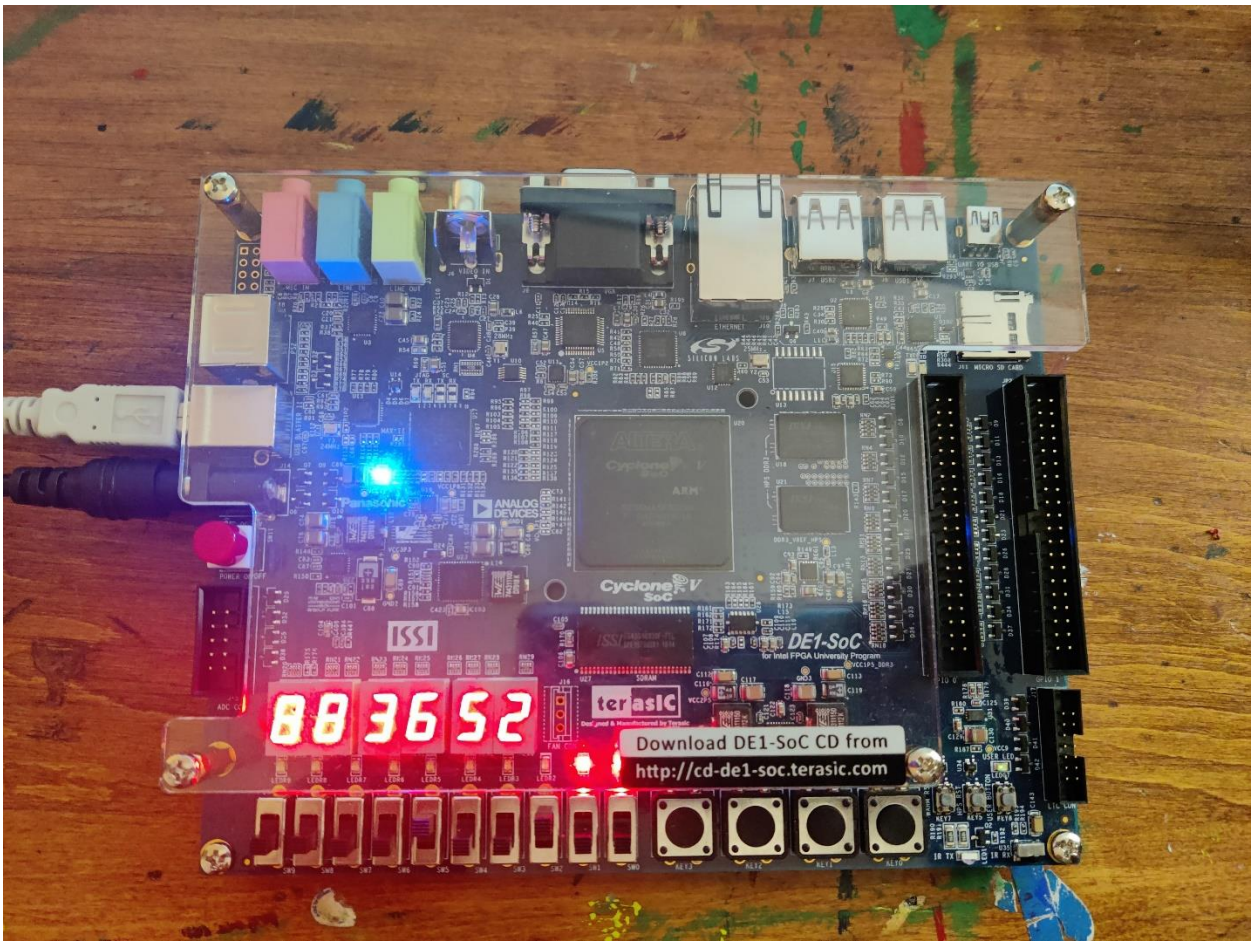


Figure: Output sample of the VHDL code

Note: All the source codes are inside lab3 folder.